DBMS PROJECT

Blood Bank & Emergency Donor-Matching System

By :

Gurjigalla Akhilesh

Department of Computer Science and Engineering

NIT Warangal

Project Overview:

The Blood Bank & Emergency Donor-Matching System is a database project built to solve a very practical problem — when a hospital urgently needs blood of a specific group, there usually isn't a quick way to check who has it, how much is available, and whether it's still usable. Most blood banks still depend on registers, spreadsheets, or a mix of both, which works fine day-to-day but breaks down exactly when speed matters most, during an emergency.

This project takes that entire process and puts it into a single relational database. Donors are registered once, and every unit of blood they give is tracked from the day it's collected to the day it expires or gets used. Hospitals are registered separately, and whenever one of them needs blood, that request is logged, matched against available stock, and reserved so the same unit can never be promised to two different requests at once.

The idea isn't just to store data — it's to make sure the right blood group reaches the right hospital before it expires or before someone else claims it first. Along the way the project also demonstrates core DBMS concepts: entity design, normalization up to 3NF, keys and constraints, and trigger-based automation that keeps inventory status accurate without anyone having to update it by hand.

The system will also:

- Keep a live count of how many units of each blood group are currently available.
- Stop the same blood unit from being reserved twice.
- Automatically flag units once they cross their expiry date.
- Give a full history of every donation, request and reservation for later auditing.

Entities:

- Donor:-

  The table represents a person who donates blood. A donor can give blood more than once, so one donor can be linked to several blood units over time.

- Hospital:-

  The table represents a hospital or medical centre that is registered to request blood from the system. Each hospital can raise multiple requests.

- Blood_Unit: -

  The table represents one physical bag of collected blood. Every unit belongs to exactly one donor and carries its own collection date, expiry date and status (Available, Reserved, Issued or Expired).

- Blood_Request:-

  The table represents an emergency or routine request raised by a hospital for a certain blood group, quantity and urgency level.

- Unit_Reservation:-

  The table that actually connects a request to specific blood units. This is what prevents one unit from being handed out to more than one hospital at the same time.

RELATIONSHIPS:

1. Donor ↔ Blood_Unit

    ○ One Donor can donate multiple Blood_Units over their lifetime. (One-to-Many)

    ○ Each Blood_Unit is collected from exactly one Donor. (Many-to-One)

2. Hospital ↔ Blood_Request

    ○ One Hospital can raise multiple Blood_Requests. (One-to-Many)

    ○ Each Blood_Request is raised by exactly one Hospital. (Many-to-One)

3. Blood_Request ↔ Unit_Reservation

    ○ One Blood_Request can reserve more than one Blood_Unit if the requested quantity is more than one. (One-to-Many)

    ○ Each Unit_Reservation is tied to exactly one Blood_Request. (Many-to-One)

4. Blood_Unit ↔ Unit_Reservation

    ○ A Blood_Unit can appear in at most one active reservation at any given time — this is what stops double booking. (One-to-One, while the reservation is active)

    ○ Each Unit_Reservation points to exactly one Blood_Unit. (One-to-One)

Functional Dependencies after Normalization

Donor Table

donor_id → first_name, last_name, date_of_birth, blood_group, rh_factor, phone, email, address, is_active, registered_at

Reason: donor_id is the Primary Key.

Hospital Table

hospital_id → name, license_number, priority_level, phone, email, address, is_active, registered_at

Reason: hospital_id is the Primary Key.

license_number → hospital_id, name, priority_level, phone, email, address, is_active, registered_at

Reason: license_number carries a UNIQUE constraint, so it also works as a candidate key.

Blood_Unit Table

unit_id → donor_id, collection_date, expiration_date, status, volume_ml, created_at

Reason: unit_id is the Primary Key.

Blood_Request Table

request_id → hospital_id, requested_blood_group, requested_rh_factor, quantity, urgency, fulfillment_status, requested_at, fulfilled_at

Reason: request_id is the Primary Key.

Unit_Reservation Table

reservation_id → request_id, unit_id, reserved_at

Reason: reservation_id is the Primary Key.

unit_id → reservation_id

Reason: unit_id has a UNIQUE constraint, since one blood unit is never allowed to sit in two active reservations at once.

Normalization

The database is normalized up to Third Normal Form (3NF).

➔ First Normal Form (1NF):

Every column holds a single value — for instance a donor's blood_group and rh_factor are kept as two separate columns instead of being crammed into one field like "O+". No table stores repeating groups of data in a single row.

➔ Second Normal Form (2NF):

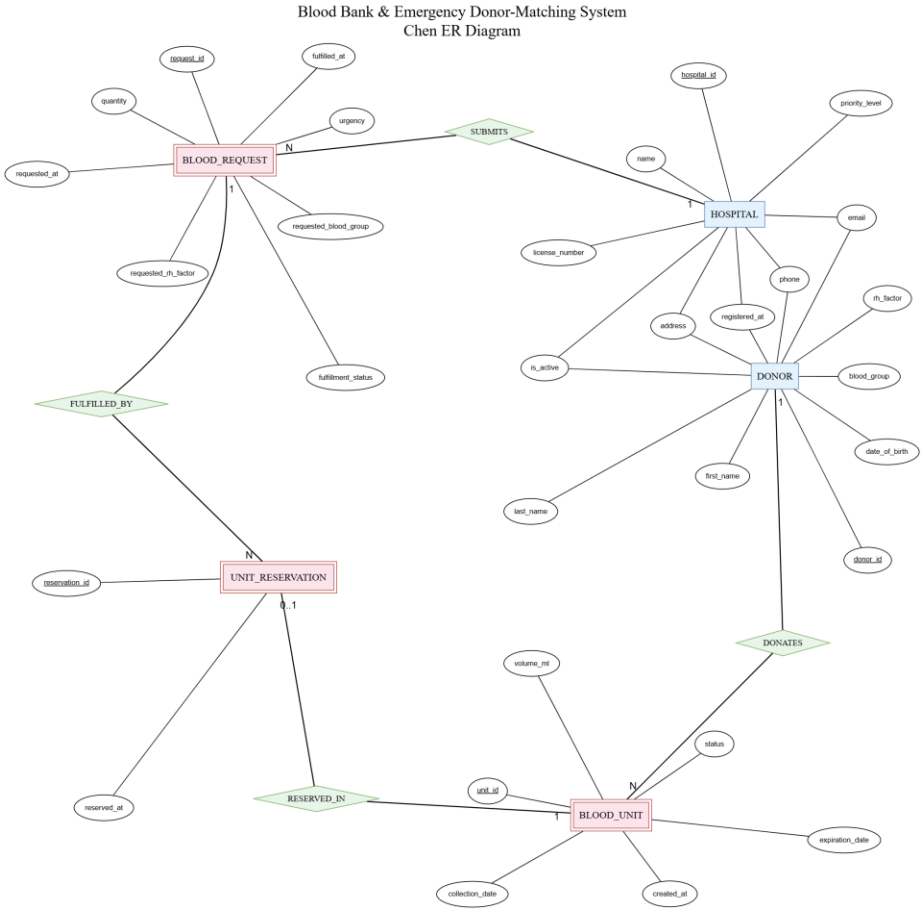
All five tables use a single-column surrogate key (donor_id, hospital_id, unit_id, request_id, reservation_id), so there's no composite key to worry about, and no attribute can partially depend on just part of a key.

➔ Third Normal Form (3NF):

There are no transitive dependencies. A hospital's name and address aren't repeated in every row of Blood_Request — they live only in the Hospital table and get pulled in through hospital_id whenever needed. The same logic applies to donor details inside Blood_Unit.

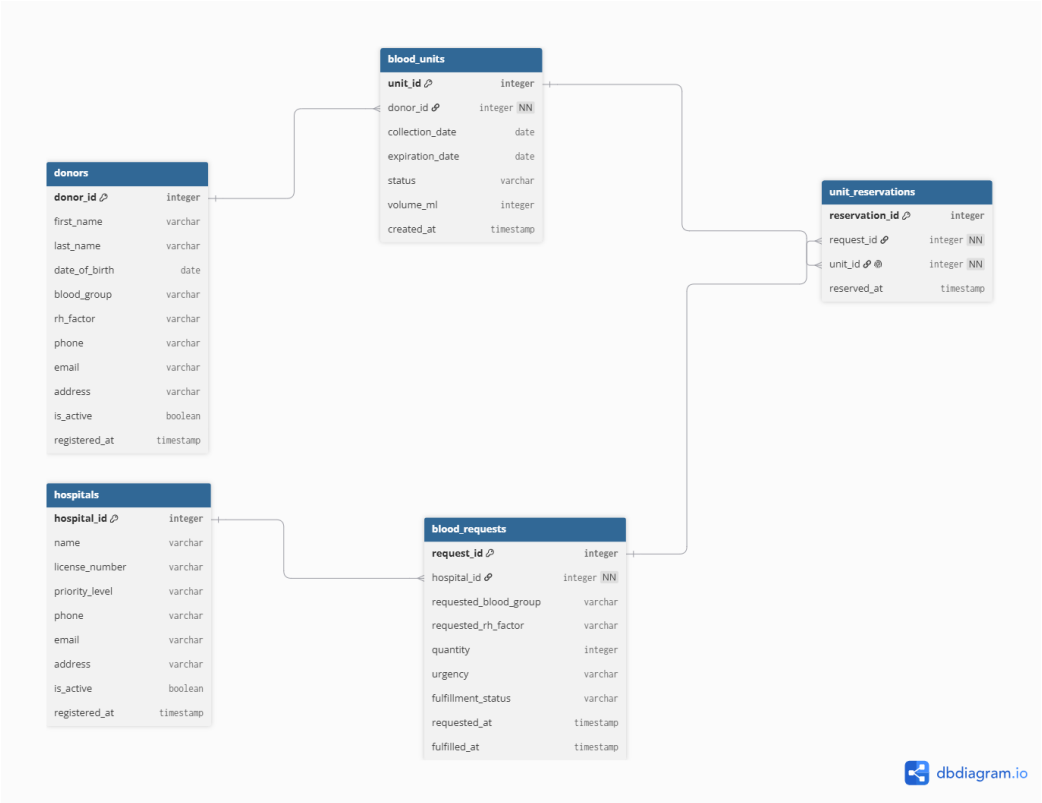
ER Diagram and Schema Diagrams

Below is the Chen-notation ER diagram for this project. Entities are drawn as rectangles, relationships as diamonds, and every attribute sits in its own oval connected to the entity it belongs to — primary keys are underlined, and the 1/N labels on each connecting line show the cardinality of that relationship.



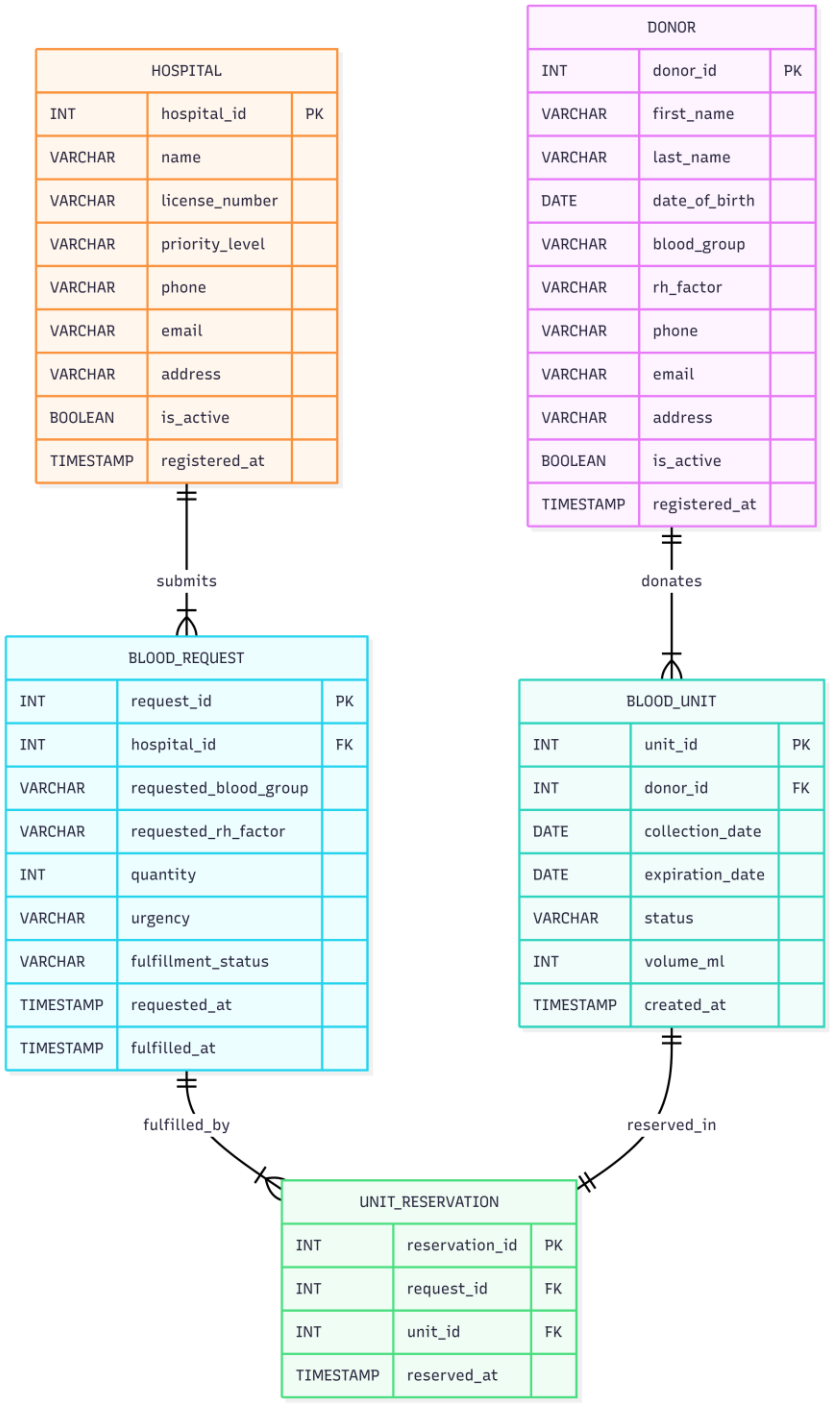
Chen ER Diagram — Blood Bank & Emergency Donor-Matching System

The next diagram is the relational schema, generated in dbdiagram.io. It shows the five tables exactly as they exist in the database — every column with its data type, the key icon marking each primary key, and the lines running between tables to show which column is a foreign key referencing which parent table.



Relational Schema Diagram (dbdiagram.io)

Finally, the crow's-foot version of the schema below expresses the same five tables using standard crow's-foot cardinality notation. It's a bit easier to read at a glance because the "one" and "many" ends of each relationship are drawn directly on the connecting line instead of being written as text, and it also marks which foreign keys are optional (0..1) versus mandatory.



Database Schema Diagram — Crow's Foot Notation

TABLE CREATION AND INSERTION:-

Hospital Table

```
CREATE TABLE Hospital(
  hospital_id INT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  license_number VARCHAR(50) UNIQUE NOT NULL,
  priority_level VARCHAR(50) CHECK (priority_level IN ('Trauma Center','General','Specialty','Clinic')),
  phone VARCHAR(15) NOT NULL,
  email VARCHAR(100),
  address VARCHAR(200),
  is_active BOOLEAN DEFAULT TRUE,
  registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
INSERT INTO Hospital (hospital_id, name, license_number, priority_level, phone, email, address, is_active,
registered_at)
VALUES
(1, 'City General Hospital', 'LIC-TS-1001', 'Trauma Center', '9440011001', 'contact@citygeneral.in', 'Abids,
Hyderabad', TRUE, '2023-01-10'),
(2, 'Sunrise Multi-Specialty Hospital', 'LIC-TS-1002', 'Specialty', '9440011002', 'info@sunrisehosp.in', 'Kondapur,
Hyderabad', TRUE, '2023-02-14'),
(3, 'Government Area Hospital', 'LIC-TS-1003', 'General', '9440011003', 'admin@gah.gov.in', 'Shamshabad,
Telangana', TRUE, '2023-03-05'),
(4, 'Rainbow Children Hospital', 'LIC-TS-1004', 'Specialty', '9440011004', 'care@rainbowch.in', 'Banjara Hills,
Hyderabad', TRUE, '2023-04-19'),
(5, 'Apex Emergency & Trauma Center', 'LIC-TS-1005', 'Trauma Center', '9440011005', 'er@apextrauma.in',
'Secunderabad, Telangana', TRUE, '2023-05-22');
```

Hospital Table

hospital_id	name	license_number	priority_level	phone	is_active
1	City General Hospital	LIC-TS-1001	Trauma Center	9440011001	1
2	Sunrise Multi-Specialty Hospital	LIC-TS-1002	Specialty	9440011002	1
3	Government Area Hospital	LIC-TS-1003	General	9440011003	1
4	Rainbow Children Hospital	LIC-TS-1004	Specialty	9440011004	1
5	Apex Emergency & Trauma Center	LIC-TS-1005	Trauma Center	9440011005	1

Donor Table

```
CREATE TABLE Donor(
  donor_id INT PRIMARY KEY,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100),
  date_of_birth DATE NOT NULL,
  blood_group VARCHAR(2) CHECK (blood_group IN ('A','B','AB','O')),
  rh_factor VARCHAR(8) CHECK (rh_factor IN ('Positive','Negative')),
  phone VARCHAR(15) NOT NULL,
  email VARCHAR(100),
  address VARCHAR(200),
  is_active BOOLEAN DEFAULT TRUE,
  registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
INSERT INTO Donor (donor_id, first_name, last_name, date_of_birth, blood_group, rh_factor, phone, email,
address, is_active, registered_at)
VALUES
(1, 'Arjun', 'Reddy', '1996-03-12', 'O', 'Negative', '9880012001', 'arjun.reddy@mail.com', 'Kukatpally, Hyderabad',
TRUE, '2024-01-05'),
(2, 'Divya', 'Sharma', '1998-07-25', 'A', 'Positive', '9880012002', 'divya.sharma@mail.com', 'Madhapur,
Hyderabad', TRUE, '2024-01-11'),
(3, 'Kiran', 'Kumar', '1990-11-02', 'B', 'Positive', '9880012003', 'kiran.kumar@mail.com', 'LB Nagar, Hyderabad',
TRUE, '2024-01-18'),
(4, 'Fathima', 'Begum', '1994-05-30', 'AB', 'Negative', '9880012004', 'fathima.b@mail.com', 'Tolichowki,
Hyderabad', TRUE, '2024-02-02'),
(5, 'Naveen', 'Chowdary', '1992-09-14', 'O', 'Positive', '9880012005', 'naveen.c@mail.com', 'Miyapur, Hyderabad',
TRUE, '2024-02-09'),
(6, 'Priya', 'Verma', '1999-01-08', 'A', 'Negative', '9880012006', 'priya.verma@mail.com', 'Begumpet, Hyderabad',
TRUE, '2024-02-20'),
(7, 'Suresh', 'Naik', '1988-06-19', 'B', 'Negative', '9880012007', 'suresh.naik@mail.com', 'Uppal, Hyderabad', TRUE,
'2024-03-01'),
(8, 'Anjali', 'Rao', '1995-12-27', 'O', 'Negative', '9880012008', 'anjali.rao@mail.com', 'Dilsukhnagar, Hyderabad',
TRUE, '2024-03-15'),
(9, 'Vikram', 'Singh', '1991-04-03', 'AB', 'Positive', '9880012009', 'vikram.singh@mail.com', 'Gachibowli,
Hyderabad', TRUE, '2024-03-28'),
(10, 'Meena', 'Iyer', '1997-08-16', 'B', 'Positive', '9880012010', 'meena.iyer@mail.com', 'Ameerpet, Hyderabad',
TRUE, '2024-04-04');
```

Donor Table (selected columns)

donor_id	first_name	last_name	blood_group	rh_factor	phone
1	Arjun	Reddy	O	Negative	9880012001
2	Divya	Sharma	A	Positive	9880012002

donor_id	first_name	last_name	blood_group	rh_factor	phone
3	Kiran	Kumar	B	Positive	9880012003
4	Fathima	Begum	AB	Negative	9880012004
5	Naveen	Chowdary	O	Positive	9880012005
6	Priya	Verma	A	Negative	9880012006
7	Suresh	Naik	B	Negative	9880012007
8	Anjali	Rao	O	Negative	9880012008
9	Vikram	Singh	AB	Positive	9880012009
10	Meena	Iyer	B	Positive	9880012010

Blood_Unit Table

```
CREATE TABLE Blood_Unit(
  unit_id INT PRIMARY KEY,
  donor_id INT NOT NULL,
  collection_date DATE NOT NULL,
  expiration_date DATE NOT NULL,
  status VARCHAR(20) DEFAULT 'Available' CHECK (status IN ('Available','Reserved','Issued','Expired')),
  volume_ml INT CHECK (volume_ml > 0),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (donor_id) REFERENCES Donor(donor_id)
);
```

```
INSERT INTO Blood_Unit (unit_id, donor_id, collection_date, expiration_date, status, volume_ml, created_at)
VALUES
(101, 1, '2026-05-01', '2026-06-12', 'Available', 450, '2026-05-01'),
(102, 2, '2026-05-02', '2026-06-13', 'Reserved', 450, '2026-05-02'),
(103, 3, '2026-05-03', '2026-06-14', 'Available', 400, '2026-05-03'),
(104, 4, '2026-05-04', '2026-06-15', 'Issued', 450, '2026-05-04'),
(105, 5, '2026-05-05', '2026-06-16', 'Available', 420, '2026-05-05'),
(106, 6, '2026-05-06', '2026-06-17', 'Available', 450, '2026-05-06'),
(107, 7, '2026-04-01', '2026-05-13', 'Expired', 450, '2026-04-01'),
(108, 8, '2026-05-08', '2026-06-19', 'Reserved', 450, '2026-05-08');
```

Blood_Unit Table

unit_id	donor_id	collection_date	expiration_date	status	volume_ml
101	1	2026-05-01	2026-06-12	Available	450
102	2	2026-05-02	2026-06-13	Reserved	450
103	3	2026-05-03	2026-06-14	Available	400
104	4	2026-05-04	2026-06-15	Issued	450
105	5	2026-05-05	2026-06-16	Available	420
106	6	2026-05-06	2026-06-17	Available	450
107	7	2026-04-01	2026-05-13	Expired	450
108	8	2026-05-08	2026-06-19	Reserved	450

Blood_Request Table

```
CREATE TABLE Blood_Request(
  request_id INT PRIMARY KEY,
  hospital_id INT NOT NULL,
  requested_blood_group VARCHAR(2) CHECK (requested_blood_group IN ('A','B','AB','O')),
  requested_rh_factor VARCHAR(8) CHECK (requested_rh_factor IN ('Positive','Negative')),
  quantity INT CHECK (quantity > 0),
  urgency VARCHAR(10) CHECK (urgency IN ('Critical','High','Medium','Low')),
  fulfillment_status VARCHAR(20) DEFAULT 'Pending' CHECK (fulfillment_status IN ('Pending','Partially
Fulfilled','Fulfilled','Cancelled')),
  requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  fulfilled_at TIMESTAMP,
  FOREIGN KEY (hospital_id) REFERENCES Hospital(hospital_id)
);
```

```
INSERT INTO Blood_Request (request_id, hospital_id, requested_blood_group, requested_rh_factor, quantity,
urgency, fulfillment_status, requested_at, fulfilled_at)
VALUES
(201, 1, 'O', 'Negative', 2, 'Critical', 'Fulfilled', '2026-06-01 08:15:00', '2026-06-01 09:00:00'),
(202, 2, 'A', 'Positive', 1, 'Medium', 'Fulfilled', '2026-06-02 10:30:00', '2026-06-02 12:10:00'),
(203, 3, 'B', 'Positive', 3, 'High', 'Partially Fulfilled', '2026-06-03 14:00:00', NULL),
(204, 1, 'AB', 'Negative', 1, 'Critical', 'Pending', '2026-06-04 05:45:00', NULL),
(205, 4, 'O', 'Positive', 2, 'Medium', 'Fulfilled', '2026-06-05 09:20:00', '2026-06-05 11:00:00');
```

Blood_Request Table

request_id	hospital_id	requested_blood_group	urgency	quantity	fulfillment_status
201	1	O	Critical	2	Fulfilled
202	2	A	Medium	1	Fulfilled
203	3	B	High	3	Partially Fulfilled
204	1	AB	Critical	1	Pending
205	4	O	Medium	2	Fulfilled

Unit_Reservation Table

```
CREATE TABLE Unit_Reservation(
  reservation_id INT PRIMARY KEY,
  request_id INT NOT NULL,
  unit_id INT NOT NULL UNIQUE,
  reserved_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (request_id) REFERENCES Blood_Request(request_id),
  FOREIGN KEY (unit_id) REFERENCES Blood_Unit(unit_id)
);
```

```
INSERT INTO Unit_Reservation (reservation_id, request_id, unit_id, reserved_at)
VALUES
(301, 201, 104, '2026-06-01 08:40:00'),
(302, 202, 102, '2026-06-02 11:00:00'),
(303, 203, 108, '2026-06-03 14:20:00'),
(304, 205, 105, '2026-06-05 10:00:00');
```

Unit_Reservation Table

reservation_id	request_id	unit_id	reserved_at
301	201	104	2026-06-01 08:40:00
302	202	102	2026-06-02 11:00:00
303	203	108	2026-06-03 14:20:00
304	205	105	2026-06-05 10:00:00

Triggers Used

Four triggers keep the inventory in sync automatically, so the status column on Blood_Unit never has to be updated by hand.

1. trg_UpdateUnitOnReservation

Fires right after a new row is added to Unit_Reservation and flips the matching blood unit's status to 'Reserved', so it can't be picked up by any other request.

```
DELIMITER $$
CREATE TRIGGER trg_UpdateUnitOnReservation
AFTER INSERT ON Unit_Reservation
FOR EACH ROW
BEGIN
  UPDATE Blood_Unit
  SET status = 'Reserved'
  WHERE unit_id = NEW.unit_id;
END$$
DELIMITER ;
```

2. trg_ReleaseUnitOnReservationDelete

If a reservation is cancelled or removed, this trigger sends the unit back to 'Available' status automatically instead of leaving it stuck as Reserved forever.

```
DELIMITER $$
CREATE TRIGGER trg_ReleaseUnitOnReservationDelete
AFTER DELETE ON Unit_Reservation
FOR EACH ROW
BEGIN
  UPDATE Blood_Unit
  SET status = 'Available'
  WHERE unit_id = OLD.unit_id AND status = 'Reserved';
END$$
DELIMITER ;
```

3. trg_PreventDuplicateReservation

This one runs before an insert even happens. It checks the status of the unit being reserved, and if that unit isn't currently 'Available', it blocks the insert and raises an error instead of letting two hospitals end up with a claim on the same bag of blood.

```
DELIMITER $$
CREATE TRIGGER trg_PreventDuplicateReservation
BEFORE INSERT ON Unit_Reservation
FOR EACH ROW
BEGIN
  DECLARE v_status VARCHAR(20);
  SELECT status INTO v_status FROM Blood_Unit WHERE unit_id = NEW.unit_id;
  IF v_status <> 'Available' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Blood unit is not available for reservation.';
  END IF;
END$$
DELIMITER ;
```

4. trg_ExpireBloodUnits

Runs before any update on Blood_Unit and checks the expiration date. If the date has already passed and the unit hasn't been issued yet, it forces the status to 'Expired' so nobody accidentally reserves blood that's no longer usable.

```
DELIMITER $$
CREATE TRIGGER trg_ExpireBloodUnits
BEFORE UPDATE ON Blood_Unit
FOR EACH ROW
BEGIN
  IF NEW.expiration_date < CURDATE() AND NEW.status <> 'Issued' THEN
    SET NEW.status = 'Expired';
  END IF;
END$$
DELIMITER ;
```

Conclusion

Building this project made it clear how much a well-structured database can simplify something that sounds complicated on paper — matching the right blood group to the right hospital before time runs out. By keeping donors, hospitals, blood units, requests and reservations as separate but connected tables, the system avoids duplicate data and makes sure every unit of blood can be traced back to the donor it came from and the request it eventually fulfilled.

The triggers turned out to be the most useful part of the whole design. Instead of relying on the front-end or a person to update stock status correctly every time, the database itself takes care of reserving, releasing and expiring blood units the moment something changes. That's really the core benefit of doing this at the database level rather than patching it together in application code later.

Overall, this project shows how the normalization, key constraints, and trigger logic covered in the DBMS course apply directly to a real situation where getting the data model right can genuinely save time when it matters most.